Frequent Itemset Mining with Local Differential Privacy

Junhui Li
Jinan University
Guangzhou, China
jhleejnu@gmail.com

Wensheng Gan*
Jinan University
Guangzhou, China
wsgan001@gmail.com

Yijie Gui
Jinan University
Guangzhou, China
y.j.gui123@gmail.com

Yongdong Wu
Jinan University
Guangzhou, China
wuyd007@qq.com

Philip S. Yu
University of Illinois at Chicago
Chicago, USA
psyu@uic.edu

ABSTRACT

With the development of the Internet, a large amount of transaction data (e.g., shopping records, web browsing history), which represents user data, has been generated. By collecting user transaction data and learning specific patterns and association rules from it, service providers can provide better services. However, because of the increasing privacy awareness and the formulation of laws on data protection, collecting data directly from users will raise privacy concerns. The concept of local differential privacy (LDP), which provides strict data privacy protection on the user side and allows effective statistical analysis on the server side, is able to protect user privacy and perform statistics on sensitive issues at the same time. This paper adopts padding-and-sampling-based frequent oracle (PSFO), combined with an interactive query-response method satisfying local differential privacy, to identify frequent itemsets in an efficient and accurate way. Therefore, this paper proposes FIML, an improved algorithm for finding frequent itemsets in the LDP setting of transaction data. The data collector generates frequent candidate sets based on the results of the previous stage and uses them for querying, and users randomize their responses in a reduced domain to achieve local differential privacy. Extensive experiments on real-world and synthetic datasets show that the FIML algorithm can find frequent itemsets more efficiently with the same privacy protection and computational cost.

CCS CONCEPTS

• **Computing methodologies → Data mining**.

KEYWORDS

differential privacy, frequent itemset, transaction database, local differential privacy.

*Corresponding author, also with Pazhou Lab, Guangzhou 510330, China

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM '22, October 17–21, 2022, Atlanta, GA, USA
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9236-5/22/10…$15.00
https://doi.org/10.1145/3511808.3557327

ACM Reference Format:
Junhui Li, Wensheng Gan, Yijie Gui, Yongdong Wu, and Philip S. Yu. 2022. Frequent Itemset Mining with Local Differential Privacy. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM '22), October 17–21, 2022, Atlanta, GA, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3511808.3557327

1 INTRODUCTION

Nowadays, with the development of big data technology and the popularization of big data analysis, it has become ubiquitous to improve the quality of service by collecting and analyzing large amounts of user data. Frequent pattern mining (FPM) [15, 21, 23] is an important and widely used data mining technology whose goal is to discover items or structures that occur frequently in user data. The discovery of interesting patterns is of great significance to business behavior and scientific research, such as mining association rules [16, 21], predicting user behavior [19, 35], and discovering correlations among different items [18, 23]. However, collecting transaction records directly from users, such as web browsing history and medical records, may reveal sensitive information about users, raising privacy concerns [2].

In the research community, differential privacy (DP) [1, 10, 17] has gradually been accepted as the de facto standard for data privacy, which provides provable privacy protection without making any assumptions about the adversary's background knowledge. In centralized settings, data collectors collect personal data from each user, randomize the dataset according to its characteristics, and publish datasets or relevant statistics that satisfy different differential privacy constraints. In this setting, the data collector can get and properly process all the user's private data. However, driven by commercial interests, data collectors may illegally trade user privacy; third parties may also be hacked or damaged, making it difficult to guarantee user privacy. Therefore, it is unrealistic to assume that data collectors or third parties are trustworthy.

Local differential privacy (LDP) [6, 9, 28] is intended for scenarios where users have complete control over their original data and do not rely on any third party. Using this technique, data collectors can collect statistics on user data while protecting the privacy of participants, without relying on the trust of the data collectors. Specifically, users process their data locally and independently using a randomization algorithm before sending it to the untrusted data collector, who then collects all the perturbed data and estimates the results of statistical analysis of the user's raw data. Data collectors can never know the exact value of sensitive data, which not only protects the

privacy of data contributors but also protects the data collectors themselves from the risk of potential data breaches. Due to its practical setup and strong privacy protection, LDP has been deployed in some popular systems. For example, Google has deployed its own LDP mechanism called RAPPOR [12, 14] into Chrome. Through RAPPOR, Google can collect users' answers to questions such as the browser's default homepage and default search engine, so as to know whether the user's browser settings have been maliciously hijacked. Apple also proposed two similar methods, CMS and HCMS [7], to perform various data collection tasks, such as identification of popular emojis, health data types, and media playback preferences in Safari, and developed SFP [27] based on CMS and HCMS to help predict spelling. Microsoft has also developed a similar system for telemetry data collection using LDP technology [8].

Assume that each user has an input value $v \in I$, where I is the input domain. Without being able to see the raw data, the data collectors want to know the distribution of input values for all users. Existing research has proposed a variety of frequency oracle (FO) protocols that data collectors can use to estimate the frequency of any particular value $x \in \mathcal{D}$. Qin *et al.* [26] considered the case where the value of each user is a set of items $v \subseteq I$, where I is the input domain. Transaction consists of a set of items, which has always been the challenging issue of research due to different applications and unequal length. To solve the problem of data mining under transaction databases, Qin *et al.* [26] proposed the "padding and sampling" technique, which processes the transactions into the same length and then samples a subset from it. FedFPM [32] adopts interactive queries between clients and servers to complete the data mining task. The sampled client responds to the query on part of its datasets with simple binary answers.

Inspired by the above research, we propose a new LDP-based transaction collection algorithm called FIML. There are three stages: In the first stage, users use FO to prune the input domain, and the data collector selects frequent 1-itemsets to construct a candidate set; in the second and third stages, the interactive query between clients and servers is used to estimate the frequency of each candidate itemset and find the frequent itemsets. Since the FIML algorithm uses the randomized response mechanism when estimating the true frequency, the overall performance under the same privacy constraints is better than the SVSM method in identifying more frequent itemsets and estimating the frequencies more accurately. The main contributions of this paper are as follows.

- This paper proposes a frequent itemset mining algorithm called FIML under the setting of local differential privacy, which can find frequent itemsets and their frequencies. The correctness and effectiveness of FIML can be rigorously verified through theoretical analysis of privacy and accuracy, as well as extensive experimental evaluation.
- FIML achieves high data utilization and incurs low communication and computation costs for each user through efficient input domain reduction and candidate generation.
- Extensive experiments on multiple real-world datasets of different scales show that the FIML algorithm has better mining performance in frequent itemset mining scenarios, outperforming other current frequent itemset mining algorithms based on LDP.

The rest of the paper is organized as follows. In Section 2, we discuss related work. Then, in Section 3, we present the background on the development of the LDP concept. In Section 4, we describe the newly proposed algorithm FIML in detail. Section 5 conducts experiments and analyzes the experimental results. Finally, we conclude this paper in Section 6.

2 RELATED WORK

2.1 Local Differential Privacy

To address privacy-preserving data mining [2], the concept of differential privacy [10] was first introduced by Dwork, mainly to solve the problem of personal privacy disclosure in the process of database analysis. Local differential privacy [9] assumes that data collectors are not trustworthy, and what users send to data collectors is not their original data, but the data perturbed by a randomization algorithm. Duchhi *et al.* [9] introduced two basic applications of LDP: mean estimation and polynomial (probability) estimation. Compared with centralized differential privacy, local differential privacy provides a stronger privacy guarantee and is more suitable for the reality that distributed users do not trust service providers in the current Internet environment. In 2016, Apple officially adopted local differential privacy to protect the privacy of iOS and MAC users, which made differential privacy famous and gradually entered the field of vision of more researchers. At present, LDP has been studied and applied in more real-world scenarios.

2.2 Heavy Hitter Identification

Estimating frequent values in the input domain is a basic mechanism of LDP. It has been extensively and well studied. Warner [33] proposed the randomized response (RR) technique that can only be used for simple binary responses. It was later generalized as the generalized randomized response (GRR) technique, which can be applied when the size of the input domain is larger than 2. Erlingsson *et al.* [12] proposed basic RAPPOR and RAPPOR, which are capable of the tasks of heavy hitter identification and overall distribution reconstruction. Bassily and Smith [4] proposed a protocol using Random Matrix Projection, in which each user randomly selects a hash function from a given family of hash functions and uses the chosen hash function to hash his value into one bit. The bit is then perturbed using the randomized response. This technique is also known as binary local hashing (BLH). Wang *et al.* [29] proposed optimal unary encoding (OUE) by minimizing the variance of unary encoding, and generalized BLH to local hashing (LH), whose output domain of the hash function family becomes $[g]$ (where $g \geqslant 2$). The optimized local hashing (OLH) is proposed by taking a partial derivative of the LH variance to get the optimal value of g. Wang *et al.* [29] observed that GRR can achieve better utility when the size of the input domain is small, and OUE or OLH works better when the size of the input domain is large.

When the input domain is large, the above methods are computationally infeasible to find heavy hitters. Therefore, in response to this situation, researchers have also proposed some feasible methods. Thakurta *et al.* [27] devised a method that is able to obtain frequent items when the dictionary is unknown. It divides the sequence into multiple segments, identifies frequent sequences at each position, and filters meaningless sequence combinations through

semantic analysis and other means to obtain complete frequent sequences. Bassily *et al.* [3] proposed a more efficient and more accurate method called TreeHist based on the technique in [27] compared to the Random Matrix Projection [4]. It first constructs a prefix tree and removes infrequent nodes by pruning, and then performs frequency estimation again on the determined candidate heavy hitters to obtain more accurate results. A non-private method [5] is combined with the heavy hitter identification idea of TreeHist [3] to construct an LDP heavy hitter identification protocol namely PrivateExpanderSketch, which achieves the optimal error in all parameters. Wang *et al.* [31] proposed the Prefix Extending Method (PEM). By grouping users, each group of users reports a prefix of their private value. This method gradually identifies longer and longer frequent prefixes and finally realizes the heavy hitter identification.

2.3 Frequent Itemset Mining

In the research problem of frequent itemset mining [15, 16, 22], each user has a transaction/record (a set of items from the input domain) and the goal is to find frequent itemsets. Itemsets are formed by combinations of items in the input domain, so the number of itemsets is exponential, and it is ineffective to encode each itemset (a power set of the original input domain) and simply apply frequency estimation. Frequent itemset mining has always been a challenging problem [20, 25]. In the setting of centralized differential privacy, many methods to deal with this kind of problem have been proposed [24, 34].

In the local setting, researchers have also proposed some solutions. Evfimievski *et al.* [13] used a pseudo-random generator to compress transaction record of indeterminate length, by carefully selecting seeds to represent a specific transaction. Qin *et al.* [26] proposed the LDPMiner protocol, which adopts a "padding and sampling" technique. The entire protocol is divided into two stages by dividing the privacy budget: obtaining a frequent candidate set and then performing frequency estimation on this set. Wang *et al.* [30] made improvements to LDPMiner and proposed the SVIM protocol, which further improved the accuracy under the same privacy guarantee. Its advantages include privacy amplification in the sampling process and the higher utility of dividing user groups than dividing privacy budgets. Wang *et al.* [30] proposed the SVSM protocol and solved the problem of frequent itemset mining for transaction data. The FedFPM framework [32] used an interactive query-response method to estimate the frequency of each candidate and find frequent patterns, which can be used for mining frequent items, itemsets, and sequences.

3 PRELIMINARIES

3.1 Problem Definition

A transaction record consists of a set of items or elements, which is a kind of unstructured data. Let $I = \{a_1, a_2, \cdots, a_d\}$ denote the item domain, and the transaction t is a subset of the domain I (i.e., $t \subseteq I$). A transaction database $\mathcal{D}$ consists of a set of transactions $\{t_1, t_2, \cdots, t_n\}$, where each transaction is a collection of arbitrary items in the domain.

Various data mining tasks can be performed on transaction databases, such as association rule mining, user behavior prediction, recommendation systems, information retrieval, and other related applications. Among them, the most widely used and most typical is frequent itemset mining. A set of items is called an itemset, and an itemset containing k items is called a k-itemset. The support count of an itemset refers to the number of occurrences of this itemset in $\mathcal{D}$, that is, the number of transactions that contain this itemset. Frequent itemsets are those whose support count is no less than the minimum support threshold.

This paper assumes that each user has one transaction/record. With LDP constraints defined on each user's data, the goal is to find itemsets that appear frequently in the user population. The problem of finding frequent itemsets can be defined as identifying top-k itemsets or finding itemsets whose occurrences are above a certain threshold, and the frequency threshold can be approximately converted to a value of k. We use the top-k version of the definition.

Definition 3.1. (top-k frequent itemsets). Given a set of transaction records $\{v^1, v^2, \cdots, v^n\} \in I^n$, an element $x \subseteq I$ is a top-k frequent itemset if its frequency $f_x = \frac{|\{v^j | x \subseteq v^j\}|}{n}$ ranks in the top k of all possible itemsets.

Frequent itemset mining is a challenging task. Note that each possible itemset can be encoded as a single value in the power set of the input domain I to form a new input domain $\mathcal{D}$. While the existing FO protocol [4] can only identify values that are very frequent in the input domain because the scale of the noise added by the FO protocol is linear with the square root of the user size n [11]. Data collectors can only get noise when any value in $\mathcal{D}$ is not frequent enough, so the FO protocol does not work well for such a large input domain $\mathcal{D}$. Although there are protocols specifically designed for large domains [31], such techniques still can not scale to the case that the binary encoding of the input domain exceeds a few hundred bits. We hope to be able to process only hundreds or thousands of itemsets. The second is that although some items and itemsets occur very frequently, each particular transaction is likely to occur relatively infrequently. Table 1 shows an example of five different transactions. Among them, no transaction occurs more than once, items a and e appear three times, c appears four times, and itemsets $\{a, c\}$ appear three times.

Table 1: Transaction database

TID	Transaction
t_1	a, b, c
t_2	a, c, d
t_3	c, d, e
t_4	a, c, e
t_5	b, e

3.2 Local Differential Privacy

Local Differential Privacy (LDP) [9] is a local model of DP for collecting user data without a trusted data collector. This paper considers a setting with multiple users and one data collector. Each user has a value v from the domain I, and the LDP mechanism M ensures that the probability of sending a value to the server is close to the probability of sending any other value.

The privacy requirement to satisfy ϵ-local differential privacy is formally defined as follows:

Definition 3.2. (ϵ-local differential privacy [9]) If an algorithm M is given, for any two inputs $v, v' \in I$ and any output $t \in Range(M)$, the inequality (1) always holds. Then, the algorithm M satisfies ϵ-local differential privacy.

$$Pr[M(v) = t] \leq e^{\epsilon} \cdot Pr[M(v') = t] \tag{1}$$

where $Range(M)$ is the set of all possible outputs of the algorithm M.

Like centralized differential privacy, local differential privacy also has the good property of sequential composition, which ensures that a series of algorithms that each satisfy LDP also satisfy LDP as a whole. That is, given a set of c algorithms $M_i(1 \leq i \leq c)$, each of which satisfies ϵ_i-LDP, the sequence composed of these c algorithms satisfies $\sum \epsilon_i$-LDP, and the value ϵ is also called the privacy budget. In addition, local differential privacy can also be combined in parallel. Given c algorithms $M_i(1 \leq i \leq c)$, each satisfying ϵ_i-LDP, and given c disjoint user groups, one can run the algorithm M_i on the i-th user group, and the whole process satisfies $max(\epsilon_i)$-LDP.

Compared to centralized settings, local differential privacy provides stronger privacy protection. Because even if the data collector is malicious, each user only reports the randomized data. The original data of each user is not leaked, and their privacy is still well protected.

3.3 Frequency Oracle

Frequency oracle (FO) [4] is able to estimate the frequency of any given value $x \in \mathcal{D}$ under LDP. It consists of a pair of algorithms $\langle M, A \rangle$, where M is used by each user to perturb the user's input value, and A is used by the data collector, which takes the reports of all users as input and can query the frequency of each value in the input domain.

3.3.1 Generalized Randomized Response. Generalized randomized response (GRR) [30] is used to deal with problems with d classifications. Everyone sends their own value with probability p and the remaining d-1 values with residual probability. A randomized response (RR) is a special case of GRR and can only be used for binary responses. The perturbation function is formally defined as:

$$\forall y \in \mathcal{D} Pr[M_{GRR(\epsilon)}(v) = y] = \begin{cases} p = \frac{e^{\epsilon}}{e^{\epsilon}+d-1}, if y = v \\ q = \frac{1}{e^{\epsilon}+d-1}, if y \neq v \end{cases} \tag{2}$$

GRR satisfies ϵ - LDP because $\frac{p}{q} = e^{\epsilon}$. The frequency of $x \in I$ is estimated according to the following equation:

$$A_{GRR(\epsilon)}(x) := \frac{C(x) - nq}{p - q} \tag{3}$$

where $C(x)$ is the number of times the value x is reported by all users, and n is the total number of users. This estimation is an unbiased estimation of the real counts of user data [29], and its variance is:

$$Var[A_{GRR(\epsilon)}(x)] = \frac{d - 2 + e^{\epsilon}}{(e^{\epsilon} - 1)^2} \cdot n \tag{4}$$

It can be seen from Formula (4) that the variance is linearly related to the size d of the input domain. Therefore, the accuracy of the GRR protocol drops rapidly when the input domain size d increases.

3.3.2 Optimized Local Hashing. Optimized local hashing (OLH) [29] first uses a random hash function to map a value of the user into a smaller domain of size g (the best choice for g is $\lceil e^{\epsilon} + 1 \rceil$), and then applies GRR to the hash values in the smaller domain $\{1, \cdots, g\}$. This can solve the problem for the larger domain with size d, where $p = 0.5$ and $q = 1/g$. In OLH, the reporting protocol is:

$$M_{OLH(\epsilon)}(v) := \left\langle H, M_{GRR(\epsilon)}(H(v)) \right\rangle \tag{5}$$

where H is chosen randomly from a family of hash functions. H maps the input domain I into the domain $\{1, \cdots, g\}$, i.e., $H: I \mapsto \{1, \cdots, g\}$, and then runs GRR on the domain $\{1, \cdots, g\}$.

For each value $x \in \mathcal{D}$, the data collector computes the number of reported values $C(x) = \left|\{j | H^j(x) = y^j\}\right|$. The data collector can then obtain an unbiased estimate of item frequency by the following equation:

$$\hat{f}_x = \frac{C(x) - n/g}{p - 1/g} \tag{6}$$

The variance of this frequency estimate is:

$$Var[A_{OLH(\epsilon)}(x)] = \frac{4e^{\epsilon}}{(e^{\epsilon} - 1)^2} \cdot n \tag{7}$$

Compared with Formula (4), the factor d - 2 + e^{ϵ} of the variance of OLH is replaced by $4e^{\epsilon}$. This shows that for smaller d (e.g., d - 2 < $3e^{\epsilon}$), the GRR is better, while for larger input domain size d, OLH can provide higher accuracy because the variance of OLH is independent of d.

3.4 Padding and Sampling Based Frequency Oracle

The LDPMiner protocol [26] addresses the challenges posed by transaction record inputs of indeterminate lengths by using a "padding and sampling" technique before applying the frequency oracle protocol for reporting. This type of protocol is called a padding-and-sampling-based frequency oracle (PSFO) protocol. The padding and sampling steps are defined as follows.

Definition 3.3. (PS [30]) The padding and sampling function requires a positive integer L as parameter, and the input is the set v $\subseteq I$. Suppose there are L dummy items $\perp_1, \perp_2, \cdots, \perp_L \notin I$. PS_L(v) does the following: if |v| < L, randomly adds $L-$|v| different dummy items to v. Then randomly selects an element v and outputs it.

Note that the input domain of the FO protocol used by PSFO is changed from I to $I \cup \{\perp_1, \perp_2, \cdots, \perp_L\}$. To estimate the frequency of an item x, you can get an estimate of the frequency of x from the FO protocol and multiply it by L. The following is the formal definition of PSFO.

Definition 3.4. (PSFO [30]). The padding-and-sampling-based frequency oracle protocol is specified by three parameters: a positive integer L, the frequency oracle protocol, and the privacy budget ϵ. It consists of a pair of algorithms: $\langle M, A \rangle$, which is defined as

follows.

$$PSFO(L, FO, \epsilon) := \left\langle M_{FO(\epsilon)}(PS_L(\cdot)), A_{FO(\epsilon)}(\cdot) \cdot L \right\rangle \tag{8}$$

By padding the transaction record to a fixed length and then sampling, the underestimation can be corrected by multiplying the count of frequency estimates by L.

3.5 Existing Work

This paper presents two state-of-the-art LDP protocols for mining frequent itemsets. The main challenges of such protocols are the unequal length of transactions and the exponential number of itemsets, which are crucial for the analysis of transactions. FedFPM [32] is a frequent pattern mining framework based on federated analysis, and its modular design can handle classical tasks for mining frequent items, itemsets, and sequences. To accomplish the task of pattern mining, FedFPM adopts an interactive query-response method between clients and the server. The server carefully uses Apriori properties and Hoeffding's inequality to generate informed queries; clients randomize their responses in a reduced pattern space to achieve local differential privacy. These responses are further aggregated on the server side to generate the next round of informed queries. The whole process runs iteratively until all frequent patterns are identified. Wang *et al.* [30] proposed the SVSM algorithm. It has five steps: First, the frequent items are mined using SVIM. Second, the data collector uses the frequent itemsets to construct the collection of candidate itemsets and sends them to users. Third, users report the number of candidate itemsets they have using the standard FO protocol; the data collector estimates the distribution of the number of candidate itemsets that users have and chooses an appropriate L, which is then sent to the user. Fourth, the user reports the itemsets in the candidate set using PSFO with the given parameter L; the data collector estimates the frequency of these candidate itemsets. Fifth, the data collector selects the top-k frequent itemsets of the candidate itemsets and uses the length distribution of user data obtained in the third step to further correct the underestimation of frequency.

Both algorithms described above have some limitations. FedFPM requires a large amount of user data for mining frequent pattern, and the mining effect may not be very good when the dataset is small. The main challenge of SVSM is appropriate choice of the padding length L,which should be set according to the distribution of user data.

4 PROPOSED METHOD

This section proposes an algorithm for Frequent Itemset Mining with the Local differential privacy (FIML). The FIML algorithm combines the advantages of both SVSM and FedFPM and solves their respective problems. Next, the key ideas of FIML and its overview are introduced, and then the implementation details of each step of the algorithm are given.

4.1 Design Overview

The design principle of FIML is to take full advantage of the two techniques of "padding and sampling" and query-response based randomized response. On the one hand, with the same privacy budget, the randomized response based on query response introduces less noise in the perturbation process, enabling data collectors to obtain more information. On the other hand, in the face of a large input domain, the padding-and-sampling based frequency oracle protocol adopted by the SVSM protocol is very advantageous because it can increase the valid user data samples to achieve higher estimation accuracy when the number of samples is insufficient. Although both of them have advantages and disadvantages, they are not compatible with each other. This is because they have different sampling domains. The former requires each user to sample and report items in their transactions, while the latter requires users to report an item that the data collector has sampled from a specified input domain, regardless of whether the item is included in their transactions.

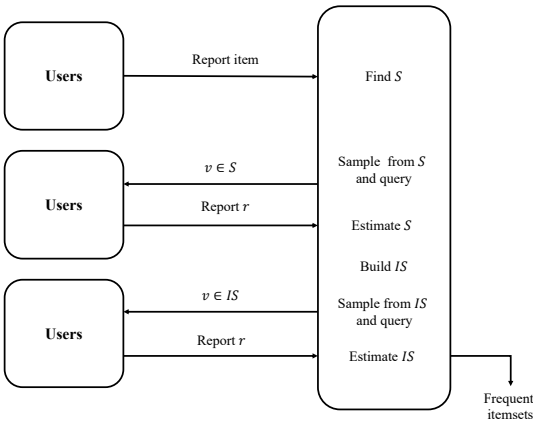

Figure 1: An overview of FIML.

Therefore, this paper designs the FIML algorithm, a solution for frequent itemset mining. It is suitable for transaction database while satisfying local differential privacy and provides better accuracy than existing protocols under the same privacy constraints. The algorithm is divided into three stages. Figure 1 shows the overview of the FIML algorithm. In the first stage, each user randomly selects an item from their private transactions, and then reports using PSFO with a small parameter L. The data collector makes a frequency estimation for each item and takes the most frequent multiple items as the collection of candidate items. In the second stage, the data collector randomly selects an item from the set of candidate items obtained in the first stage and sends it to the user, and the item being sent is random for each user. Upon receipt of a candidate item, each user generates a binary response based on whether the transaction contains the item, with 1 indicating yes and 0 indicating no, and reports it using the randomized response technique. The data collector estimates the true frequency of these candidate items based on the information collected. In the third stage, the data collector selects the most frequent items and builds the set of candidate itemsets. Then, frequency estimation for each candidate itemset is performed using a method similar to that used in the second stage. The algorithm effectively combines the advantages of the padding-and-sampling-based frequency oracle technology and the query-response-based randomized response technology. Thus, each possible frequent itemset can obtain more estimated samples. Extensive experiments have also demonstrated its effectiveness.

4.2 Implementation Detail

In this paper, we do not adopt the method of dividing the privacy budget, but we group users to obtain a better effect for frequent itemset mining. FIML divides users into three groups, each of which participates in one task. Since the privacy budget required by each task is ϵ, each user is protected by ϵ - LDP.

4.2.1 Prune the Domain. When the input domain is large (e.g., tens of thousands), it is unwise to estimate the frequency directly, because there may be no frequent items in the input domain. The first thing the data collector should do is to prune the domain and make it focus on a smaller candidate set for more efficient frequency estimation. Specifically, in this phase, each user randomly selects an item from their private transaction to report, with the length limit L set to 1:

$$M_{PSFO(1,OLH,\epsilon)}(v) \tag{9}$$

Setting $L = 1$ has two advantages. First, there is no need to obtain the exact value of L, so no additional privacy budget or user data is required. Second, each user only needs to report one item in the transaction. Although the actual frequency will be underestimated, this underestimation is overall and will not have an additional impact on the ranking of each item in the entire input domain. Therefore, the FIML algorithm can obtain relatively accurate candidate frequent items.

After receiving responses from all users, the data collector estimates the frequency of each value in the input domain using the following data collection function:

$$A_{PSFO(1,OLH,\epsilon)}(x) \tag{10}$$

After obtaining the frequency, the data collector selects the items that are more frequent and obtains a set S containing the $1.5k$ most frequent items (tested, $1.5k$ basically includes all the top-k frequent items). Note that when the size of the original input domain is close to or less than $1.5k$, this step can be omitted.

4.2.2 Candidate Frequency Estimation. After calculating the set of candidate items, for each user, the data collector randomly selects an item $v \in S$ and sends it. After receiving the candidate items, each user determines whether the candidate item is included in their private transactions, and generates a corresponding binary bit b, if it is included, $b = 1$; otherwise $b = 0$. Then, each user performs the randomized response technique on this binary bit b to achieve local differential privacy.

Algorithm 1 describes how the user generates a binary response. After the user receives the candidate sent from the data collector, he/she immediately checks whether the local private transaction contains the candidate (as shown in line 1, where $\mathbb{I}(\cdot)$ represents the indicator function, if the expression in parentheses is true, then returns 1; otherwise returns 0). The user then uses randomized response techniques to randomize its generated bit b locally (as shown in line 2) before sending the randomized response to the data collector (as shown in line 3). In this algorithm, the user only needs to judge whether the candidate is in the local transactions and randomize the response bits. Compared with SVSM, the calculation amount of each user is greatly reduced, and the amount of communication between the user and the data collector has also been reduced to a certain extent.

Algorithm 1: User response

Input: local transaction: v; the privacy budget: ϵ; a candidate: c.
Output: response: r.

1 $b \leftarrow \mathbb{I}(c \subseteq v)$;
2 $r \leftarrow$ randomize b with (2);
3 **return** r

In the FIML algorithm, the interaction scheme between the data collector and the user is carefully designed. The user only needs one bit to respond to the candidate query of the data collector, which is much simpler than the interaction method of the existing frequent itemset mining algorithm. Therefore, FIML only needs to add less response noise to achieve the same local differential privacy guarantee when estimating candidate items and candidate itemsets. This can help the algorithm obtain higher data utilization.

Algorithm 2: Data collector statistic

Input: collection of randomized response: R; the privacy budget: ϵ; a candidate set: S
Output: top-k itemset: S'; the frequency of top-k items: F'

1 $F' \leftarrow \{\}$
2 **for** candidate c in S **do**
3 | the data collector calculates and calibrates the frequency F'_c of c by (3);
4 **end**
5 the data collector selects the top-k candidates to form S' according to the estimated frequency;
6 **for** candidate c in S' **do**
7 | $F' = F' \cup \{F'_c\}$;
8 **end**
9 **return** S' and F'

Algorithm 2 describes how the data collector processes binary randomized response data reported by users. The data collector collects binary randomized responses for each candidate item and estimates the frequency of each candidate item over all the collected data (Lines 2-3). According to the estimated frequency, the most frequent k items are selected to form the collection of top-k frequent items, denoted as S', and the frequency of the corresponding top-k frequent items is added to the set F' (Lines 4-6).

4.2.3 Building Candidate Itemsets. The most challenging part of frequent itemset mining is constructing the set IS of candidate itemsets because the number of potentially frequent itemsets can be exponential. Thus, if we can reduce this to a manageable range (thousands), we can turn the problem into frequent item mining and run an algorithm similar to the second stage.

Suppose S' is the k most frequent items calculated in the second stage. In order to avoid estimating an exponential number of candidate itemsets, this paper uses the frequency estimation information of S' to reduce the candidate itemsets. For an itemset x, FIML first calculates its frequency, denoted as f_{x}, as the product of the estimated values of all its items, that is, $f_{\mathrm{x}} = \prod_{x \in \mathrm{x}} A'(x)$. Due

to the noise introduced by the randomized algorithm, although the frequency estimation in the second stage is unbiased, the estimation of a single frequency may still be between 0 and 1. Therefore, the algorithm needs to normalize the estimated frequency of frequent items. Moreover, it is unlikely for the frequency of the frequent itemset and the frequency of its true subsets to be equal. Thus, after normalization, the frequency of any itemset is multiplied by a factor of 0.9, so that their frequency is between 0 and 1 when guessing the frequency of itemsets, i.e. $A'(x) = \frac{0.9*(A(x)-a)}{max_{x \in S'} A(x)-a}$, where a is the minimum possible value of the estimated frequency using randomized response techniques, and is - $\frac{1}{e^{\epsilon}-1}$. Then the data collector selects $1.5k$ itemsets with the highest guessing frequency to construct IS. The intuition is that frequent itemsets are unlikely to be composed of several infrequent items, but the itemset composed of frequent items may be infrequent. However, since the combination of frequent items is exponential, after weighing this, this paper decides to use the known information to guess the frequency of itemsets, which is an effective metric to measure whether each itemset is likely to be frequent.

IS is constructed as follows:

$$IS := \{\mathrm{x} : \mathrm{x} \subseteq S, 1 < |\mathrm{x}| < log_2 k, \prod_{x \in \mathrm{x}} A'(x) > t\} \tag{11}$$

where $|IS| = 1.5k$ by choosing an appropriate t.

4.2.4 Mining Frequent Itemsets. This step is similar to the second step. After obtaining the collection of candidates IS, for each user, the data collector randomly selects an itemset $v \in IS$ and then sends it to the users. After receiving each candidate, the user will determine whether the candidate itemset is a subset of the local transaction and generate a binary response b if it is a subset, $b = 1$; otherwise $b = 0$. The user then performs randomized response techniques on this binary response and reports it to the data collector to achieve local differential privacy. How the user generates the binary response has been described in detail in Algorithm 1.

The data collector collects the binary random responses of each itemset, and uses the collected information to estimate the frequency of each candidate itemset. The k itemsets that are most frequent are selected as the final result. Algorithm 2 also describes this in detail. So far, the mining of frequent itemsets has been completed.

4.3 Privacy Analysis

Local differential privacy has two good properties: sequential composition and parallel composition properties [11]. Through the use of these two properties, it is easy to prove that the FIML algorithm satisfies local differential privacy. The FIML algorithm does not divide the privacy budget but divides the user population into three groups. The first group uses the optimized local hashing protocol to report, while the second and third groups use the randomized response protocol to response to the data collector's query.

Theorem 4.1. *The FIML algorithm satisfies ϵ-local differential privacy.*

Proof. This paper first proves that the generalized randomized response (GRR) technique conforms to local differential privacy. Based on the definition of LDP, it can be seen that

$$\frac{Pr[M(v_i) = y]}{Pr[M(v_j) = y]} \leq e^{\epsilon} \tag{12}$$

where v_i and v_j are any possible input values. In generalized randomized response techniques, there are only two options for the probability of any output: $\frac{1}{e^{\epsilon}+d-1}$ and $\frac{e^{\epsilon}}{e^{\epsilon}+d-1}$. Therefore, we have the following inequality:

$$\frac{Pr[M(v_i) = y]}{Pr[M(v_j) = y]} \leq \frac{max\{Pr[M(v) = y]\}}{min\{Pr[M(v) = y]\}} = e^{\epsilon} \tag{13}$$

Therefore, the GRR satisfies local differential privacy. And the RR used in this paper, as a special case of the GRR, also satisfies local differential privacy. Next, we prove that optimal local hashing (OLH) satisfies local differential privacy. For any two inputs v_i andv_j, and any output $\langle H, y \rangle$, we have

$$\frac{Pr[\langle H, y \rangle | v_i]}{Pr[\langle H, y \rangle | v_j]} = \frac{Pr[M_{GRR}(H(v_i)) = y]}{Pr[M_{GRR}(H(v_j)) = y]} \leq e^{\epsilon} \tag{14}$$

Therefore, OLH is consistent with local differential privacy, and the method used in each stage of the FIML algorithm satisfies ϵ-local differential privacy. Because the user groups used in the three stages do not intersect with each other, it can be seen from the parallel composition principle of local differential privacy that the whole algorithm satisfies ϵ-local differential privacy.

□

5 PERFORMANCE EVALUATION

In this section, several experiments are designed to discuss and verify the effectiveness of the FIML algorithm and other existing methods. The experiments mainly focus on two aspects. 1) How many frequent itemsets can be recognized by FIML and how accurate is the frequency estimation of these itemsets? 2) How much does FIML improve over other existing methods, especially for SVSM?

5.1 Setup of Experiments

All algorithms were implemented in Python 3.7 and performed on a personal computer running a Windows 10 system, with an i5-8250U core, 1.60GHz CPU, and 8 GB of free RAM. Each experiment was performed 10 times, and we reported the average value.

Datasets. We prepared four real-world datasets for evaluating the proposed FIML algorithm. The features of each dataset are shown in Table 2. Chess is a dense database from the UCI chess dataset, although it contains only 3196 transactions. Connect is prepared and based on the UCI connect-4 dataset. Accidents and OnlineRetail are two large datasets, and they are made up of anonymized traffic accident data and online retail data, respectively.

Table 2: Characteristics of different datasets

Dataset	Trans.	Items	AvgLen
Chess	3196	75	37
Connect	67557	129	43
Accidents	340183	468	33.8
OnlineRetail	541909	2603	4.37

Evaluation and metrics. This paper uses the following metrics to measure the utility of the FIML algorithm. Let the top-k frequent itemsets of the original dataset be $X_t = \{x_1, x_2, x_3, \ldots, x_k\}$, where x_i is the i-th frequent itemset. The set of k frequent itemsets found by the LDP algorithm is denoted as X_r. Then, $X_t \cap X_r$ is the set of the top-k itemsets found by FIML.

Normalized cumulative rank (NCR) [31]: this metric assigns each itemset a quality function $q(.)$ and uses the normalized cumulative gain (NCG), where $NCG = \sum_{x \in X_r} q(x)$. We utilize the ranking of the frequency of x in the real-world dataset to instantiate the quality function. Details are as follows: The highest ranked itemset has a score of k, that is, $q(x_1) = k$. Then, the second ranked itemset has a score of k - 1. For the k-th ranked itemset, the score is 1. However, all other itemsets have a score of 0. The sum of scores (NCG) is then divided by the maximum possible score, where $\sum_{x \in X_t} q(x) = k(k+1)/2$. In this way, we can normalize it to a value from 0 to 1. Therefore, we give the formula of the normalized cumulative rank (NCR) as $NCR = \frac{\sum_{x \in X_r} q(x)}{\sum_{x \in X_t} q(x)}$.

Mean squared error (MSE) [30]: it is used to measure the accuracy of the frequency estimation, where $MSE = \frac{1}{|X_r \cap X_t|} \sum_{x \in X_r \cap X_t} (n \cdot f_x - A(x))^2$. It is worth noting that the mean squared error only considers frequent itemsets that the algorithm found successfully.

Note. In order to implement the FIML algorithm, we design the following scheme: 1/3 of all users are assigned to find the candidates for frequent items. After that, another 1/3 of the users are assigned to estimate the frequency of the candidates and find top-k frequent items. Then, we use these frequent items and their estimated frequencies to build a candidate set IS. The remaining users are arranged to mine frequent itemsets and estimate their frequency. In order to compare the effects of the FIML and SVSM algorithms, we instantiate the SVSM according to [30].

5.2 Comparison of Effectiveness under Different Privacy Budgets

From Figure 2(a) to Figure 5(a), we can see the expected phenomenon. When $k = 64$, as the increase of the privacy budget, the quality of the frequent itemsets found by the two algorithms has an upward trend in all datasets. Because the higher the privacy budget is, the more privacy information users provide. Most importantly, data collectors can mine more frequent itemsets of good quality. However, we can still see that in the chess dataset, when $k = 64$, $\epsilon = 2$ or $k = 64$, $\epsilon = 3$, the value of the NCR is smaller than that of the previous stage. This is because the FIML algorithm groups users instead of dividing the privacy budget ϵ. During the process of grouping users, the distribution of each group of users may be different, and this way causes the fluctuation in data. In general, when the privacy budget is small, the increase of the privacy budget can bring about a greater improvement in the quality of the itemsets. When the privacy budget is large, the quality of the excavated itemsets tends to be stable. Besides, we also observe that the quality of the frequent itemsets found by the proposed method is better than that of the SVSM algorithm in the range of the privacy budget from 1 to 4. This further verifies that the new algorithm has higher data utility and can mine the frequent itemsets more effectively under the same privacy budget.

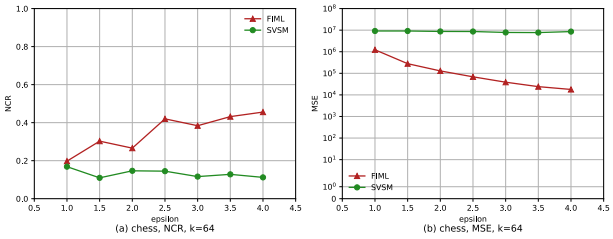

Figure 2: Impact of privacy budget (Chess).

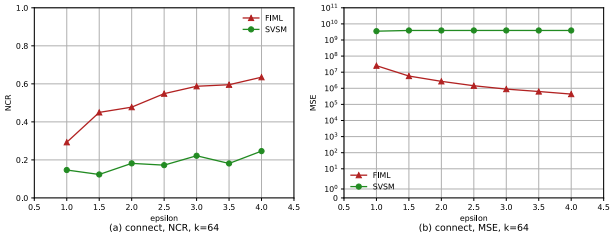

Figure 3: Impact of privacy budget (Connect).

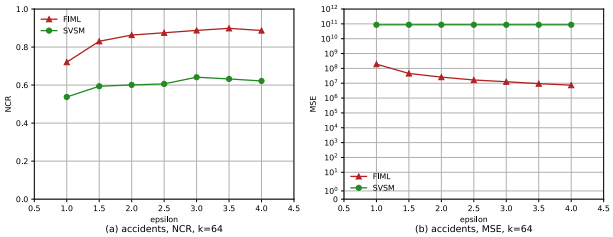

Figure 4: Impact of privacy budget (Accidents).

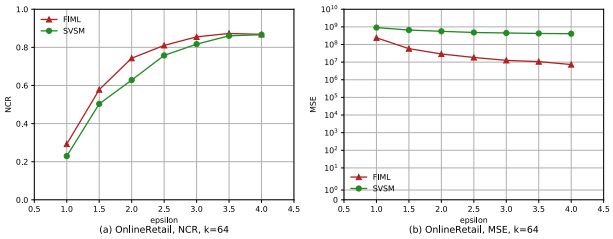

Figure 5: Impact of privacy budget (OnlineRetail).

It can be seen that from Figure 2(b) to Figure 5(b), when the privacy budget increases from 1 to 4, the MSE value shows a downward trend in all testing datasets. This verifies that the larger the privacy budget is, the more information the data collector has on the user data. Then, we can get a more accurate frequency estimate with a smaller MSE. At the same time, compared with the SVSM algorithm, the MSE value of the FIML algorithm is much smaller. In particular, the degree of reduction varies according to the characteristics of the dataset.

5.3 Comparison of Effectiveness under Different k Values

As can be seen from Figure 6(a) to Figure 9(a), the quality of the frequent itemsets mined by FIML is better than that of SVSM under different k values. When k is small, the fluctuation of the NCR value on those itemsets mined by the two algorithms is large and the quality of the itemsets is unstable. For example, when the value of k is from 5 to 50, we can see this phenomenon in the Accidents dataset. This is because the local differential privacy mechanism introduces noise. When k is small, the error rate of top-k frequent itemset mining is low, and it is prone to misjudgment of frequent items. On the other hand, when k is large, with the increase of k, the quality of the frequent itemsets that can be mined by these two algorithms has a downward trend. This is because the increase of k makes the candidate larger and the samples obtained by each candidate decrease, which leads to the decrease of the accuracy of the frequency estimation of the two algorithms for each candidate.

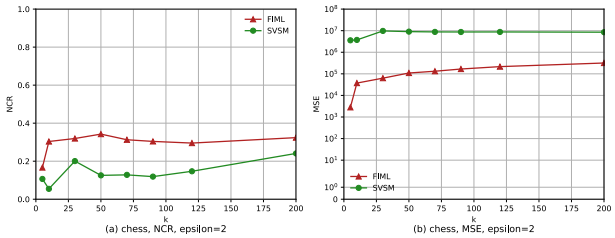

Figure 6: Impact of k value (chess).

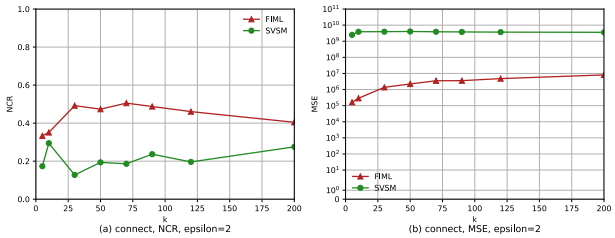

Figure 7: Impact of k value (connect).

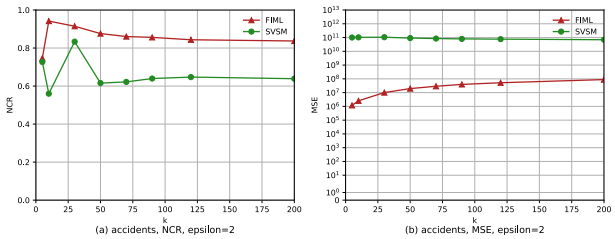

Figure 8: Impact of k value (accidents).

From Figure 6(b) to Figure 9(b), we can see that the MSE value of the proposed FIML method for estimating the frequency of the

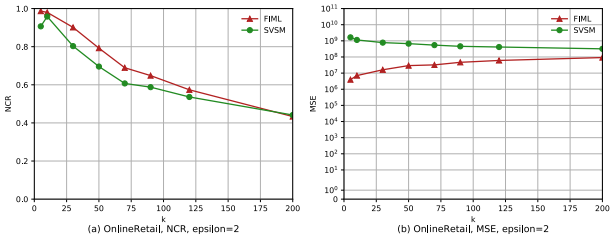

Figure 9: Impact of k value (OnlineRetail).

itemset is lower than that of the SVSM method. Notice that the specific situation varies according to different datasets. Therefore, the new method can obtain more accurate frequency estimates than SVSM. It can also be observed that, with the increase of the value of k, the MSE value of FIML increases, which is due to the reduction of the samples when estimating candidates. However, because the padding-and-sample-based frequency oracle adopted by SVSM in the latter two stages can obtain more samples than the randomized response technology based on query and response used in FIML, when the value of k is large, SVSM can obtain more samples. The accuracy of the frequency estimation of the itemsets is improved and the MSE is decreased slightly.

6 CONCLUSION

In this paper, the local differential privacy problem for frequent itemset mining from a transaction database is studied. This problem is somewhat challenging, and we proposed the FIML algorithm for solving it. FIML utilizes PSFO for the construction of candidate sets, which allows users to randomly choose an item from the transaction data of indeterminate length for reporting. In addition, it adopts the query-response based randomized response technology. According to the frequency estimation in the previous stage, the data collector carefully constructs the candidate set and generates the query, and then the user performs the randomized response technology on the real response based on the local transaction data to achieve local differential privacy. With the improvements above, the FIML algorithm has higher data utilization and lower communication costs. Its performance is better than other existing algorithms for this research problem, and it is more suitable for its application in distributed scenarios. Finally, we theoretically prove that the FIML algorithm satisfies local differential privacy, and through experiments on a wide range of datasets, we prove that our proposed method is effective.

ACKNOWLEDGMENTS

This research was supported in part by the National Natural Science Foundation of China (Grant Nos. 62002136 and 61932011), Guangzhou Basic and Applied Basic Research Foundation (Grant No. 202102020277), Guangdong Basic and Applied Basic Research Foundation (Grant Nos. 2019B1515120010 and 2022A1515011861), Guangdong Key R&D Plan2020 (Grant No. 2020B0101090002), and National Key R&D Plan2020 (Grant No. 2020YFB1005600).

REFERENCES

[1] Martin Abadi, Andy Chu, Ian Goodfellow, H Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. Deep learning with differential privacy. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. 308–318.

[2] Rakesh Agrawal and Ramakrishnan Srikant. 2000. Privacy-preserving data mining. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 439–450.

[3] Raef Bassily, Kobbi Nissim, Uri Stemmer, and Abhradeep Guha Thakurta. 2017. Practical locally private heavy hitters. *Advances in Neural Information Processing Systems* 30 (2017).

[4] Raef Bassily and Adam Smith. 2015. Local, private, efficient protocols for succinct histograms. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing*. 127–135.

[5] Mark Bun, Jelani Nelson, and Uri Stemmer. 2019. Heavy hitters and the structure of local privacy. *ACM Transactions on Algorithms* 15, 4 (2019), 1–40.

[6] Graham Cormode, Somesh Jha, Tejas Kulkarni, Ninghui Li, Divesh Srivastava, and Tianhao Wang. 2018. Privacy at scale: Local differential privacy in practice. In *Proceedings of the International Conference on Management of Data*. 1655–1658.

[7] differential privacy team at Apple. 2017. Learning with privacy at scale. https://machinelearning.apple.com/research/learning-with-privacy-at-scale

[8] Bolin Ding, Janardhan Kulkarni, and Sergey Yekhanin. 2017. Collecting telemetry data privately. *Advances in Neural Information Processing Systems* 30 (2017).

[9] John C Duchi, Michael I Jordan, and Martin J Wainwright. 2013. Local privacy and statistical minimax rates. In *IEEE 54th Annual Symposium on Foundations of Computer Science*. IEEE, 429–438.

[10] Cynthia Dwork. 2008. Differential privacy: A survey of results. In *International conference on theory and applications of models of computation*. Springer, 1–19.

[11] Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Foundations and Trends in Theoretical Computer Science* 9, 3-4 (2014), 211–407.

[12] Úlfar Erlingsson, Vasyl Pihur, and Aleksandra Korolova. 2014. Rappor: Randomized aggregatable privacy-preserving ordinal response. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. 1054–1067.

[13] Alexandre Evfimievski, Johannes Gehrke, and Ramakrishnan Srikant. 2003. Limiting privacy breaches in privacy preserving data mining. In *Proceedings of the Twenty-Second ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. 211–222.

[14] Giulia Fanti, Vasyl Pihur, and Úlfar Erlingsson. 2015. Building a RAPPOR with the unknown: Privacy-preserving learning of associations and data dictionaries. *arXiv preprint, arXiv:1503.01214* (2015).

[15] Philippe Fournier-Viger, Wensheng Gan, Youxi Wu, Mourad Nouioua, Wei Song, Tin Truong, and Hai Duong. 2022. Pattern mining: Current challenges and opportunities. In *International Conference on Database Systems for Advanced Applications*. Springer, 34–49.

[16] Philippe Fournier-Viger, Jerry Chun-Wei Lin, Bay Vo, Tin Truong Chi, Ji Zhang, and Hoai Bac Le. 2017. A survey of itemset mining. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 7, 4 (2017), e1207.

[17] Arik Friedman and Assaf Schuster. 2010. Data mining with differential privacy. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 493–502.

[18] Wensheng Gan, Jerry Chun-Wei Lin, Han-Chieh Chao, and Justin Zhan. 2017. Data mining in distributed environment: a survey. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 7, 6 (2017), e1216.

[19] Wensheng Gan, Jerry Chun-Wei Lin, Philippe Fournier-Viger, Han-Chieh Chao, and Hamido Fujita. 2018. Extracting non-redundant correlated purchase behaviors by utility measure. *Knowledge-Based Systems* 143 (2018), 30–41.

[20] Wensheng Gan, Jerry Chun-Wei Lin, Philippe Fournier-Viger, Han-Chieh Chao, Tzung-Pei Hong, and Hamido Fujita. 2018. A survey of incremental high-utility itemset mining. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 8, 2 (2018), e1242.

[21] Wensheng Gan, Jerry Chun-Wei Lin, Philippe Fournier-Viger, Han-Chieh Chao, Vincent S Tseng, and Philip S Yu. 2021. A survey of utility-oriented pattern mining. *IEEE Transactions on Knowledge and Data Engineering* 33, 4 (2021), 1306–1327.

[22] Wensheng Gan, Jerry Chun-Wei Lin, Philippe Fournier-Viger, Han-Chieh Chao, and Justin Zhan. 2017. Mining of frequent patterns with multiple minimum supports. *Engineering Applications of Artificial Intelligence* 60 (2017), 83–96.

[23] Jiawei Han, Hong Cheng, Dong Xin, and Xifeng Yan. 2007. Frequent pattern mining: current status and future directions. *Data Mining and Knowledge Discovery* 15, 1 (2007), 55–86.

[24] Ninghui Li, Wahbeh Qardaji, Dong Su, and Jianneng Cao. 2012. Privbasis: Frequent itemset mining with differential privacy. *arXiv preprint arXiv:1208.0093* (2012).

[25] José María Luna, Philippe Fournier-Viger, and Sebastián Ventura. 2019. Frequent itemset mining: A 25 years review. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 9, 6 (2019), e1329.

[26] Zhan Qin, Yin Yang, Ting Yu, Issa Khalil, Xiaokui Xiao, and Kui Ren. 2016. Heavy hitter estimation over set-valued data with local differential privacy. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*. 192–203.

[27] Abhradeep Guha Thakurta, Andrew H Vyrros, Umesh S Vaishampayan, Gaurav Kapoor, Julien Freudiger, Vivek Rangarajan Sridhar, and Doug Davidson. 2017. Learning new words. US Patent 9,594,741.

[28] Ning Wang, Xiaokui Xiao, Yin Yang, Jun Zhao, Siu Cheung Hui, Hyejin Shin, Junbum Shin, and Ge Yu. 2019. Collecting and analyzing multidimensional data with local differential privacy. In *IEEE 35th International Conference on Data Engineering*. IEEE, 638–649.

[29] Tianhao Wang, Jeremiah Blocki, Ninghui Li, and Somesh Jha. 2017. Locally differentially private protocols for frequency estimation. In *26th USENIX Security Symposium*. 729–745.

[30] Tianhao Wang, Ninghui Li, and Somesh Jha. 2018. Locally differentially private frequent itemset mining. In *IEEE Symposium on Security and Privacy*. IEEE, 127–143.

[31] Tianhao Wang, Ninghui Li, and Somesh Jha. 2019. Locally differentially private heavy hitter identification. *IEEE Transactions on Dependable and Secure Computing* 18, 2 (2019), 982–993.

[32] Zibo Wang, Yifei Zhu, Dan Wang, and Zhu Han. 2022. FedFPM: A unified federated analytics framework for collaborative frequent pattern mining. In *IEEE International Conference on Computer Communications*. IEEE, 1–10.

[33] Stanley L Warner. 1965. Randomized response: A survey technique for eliminating evasive answer bias. *J. Amer. Statist. Assoc.* 60, 309 (1965), 63–69.

[34] Chen Zeng, Jeffrey F Naughton, and Jin-Yi Cai. 2012. On differentially private frequent itemset mining. *The VLDB journal: Very Large Data Bases: a Publication of the VLDB Endowment* 6, 1 (2012), 25.

[35] Chunkai Zhang, Zilin Du, Yuting Yang, Wensheng Gan, and Philip S Yu. 2021. On-shelf utility mining of sequence data. *ACM Transactions on Knowledge Discovery from Data* 16, 2 (2021), 1–31.